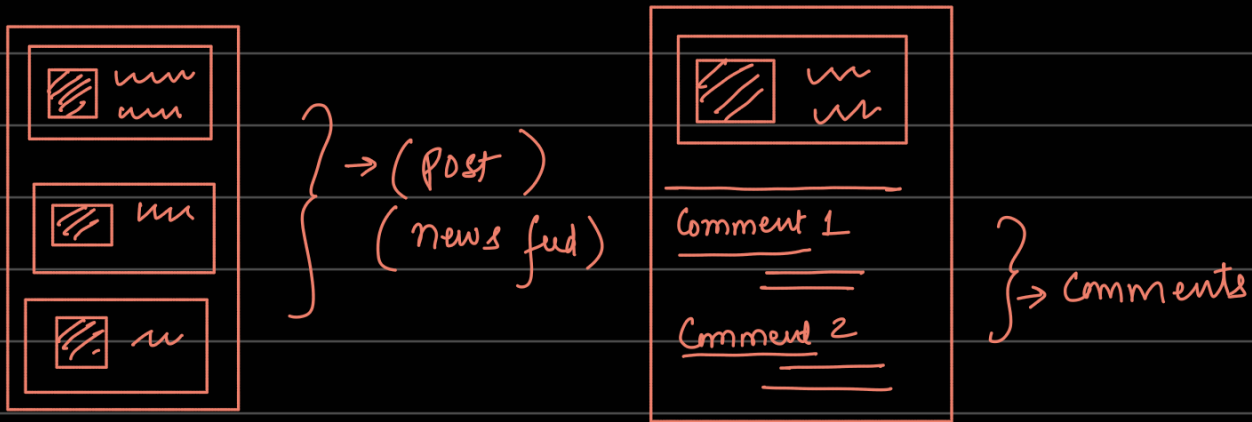


POST APPS

- 1) Facebook 2) Instagram 3) twitter

what we want?



Objective

- 1) following — followers
- 2) quickly load (posts — news feeds)
- 3) low latency
- 4) privacy types → (not so relevant)
- 5) comments → (infinitely nested)

(Reads >> writes)

Capacity Estimation

- 1) 100 char a post — 100 bytes for text + (100 byte meta data)
- 2) 1 billion post / per day = 1 billion
x 200 bytes → (post + metadata)
x 365 = 73 Tb
- 3) average follow . → 100
- 4) Comment length → 100 bytes , 100 bytes of metadata
⇒ 200 bytes

5) most comments on one post = 1 million, 1 million · 200 byte

= 200 mb

FETCHING → FOLLOWER / FOLLOWING

get followers (userid) } → we want it to be fast.
get following (userid)

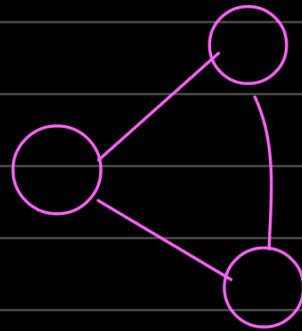
Which DB

↓
mysql

followers	following	id
1	2	
1	3	
4	1	

↓
(no-sql) (cassandra)

1) leader less replication



1) problem Partition

followers / following / followers / following

1	2
1	3
4	1

7	1
---	---

[1) write can happen on any node and then it will transfer to another node.

problem

1) Dirty Reads

→ getfollowing(1) → {2, 3} (fast)

getfollowers(1) → {4}, {7}

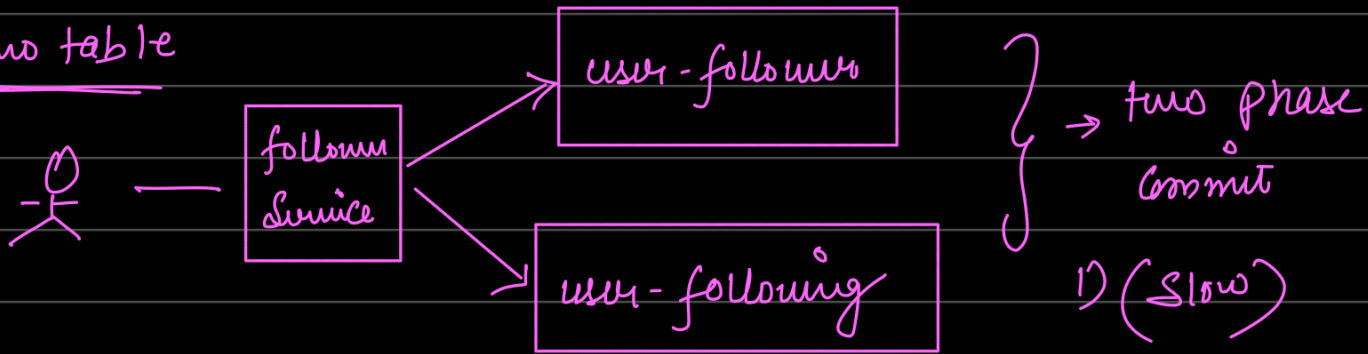
(Slow)

(combine by service)

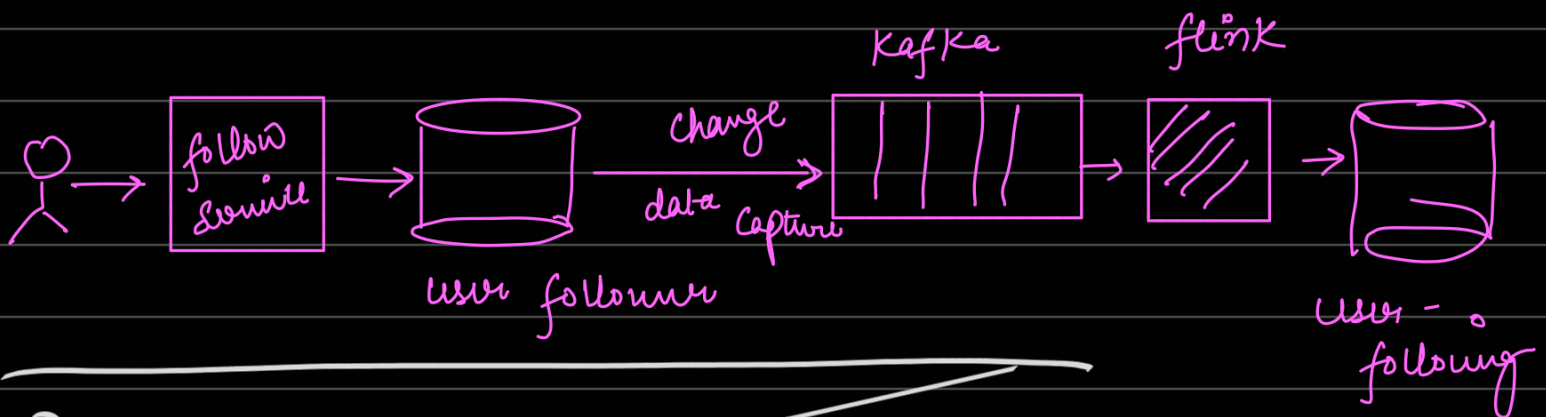
↓
{ 4, 7 }

Possible Solution

1) two table



2) two table - with streaming

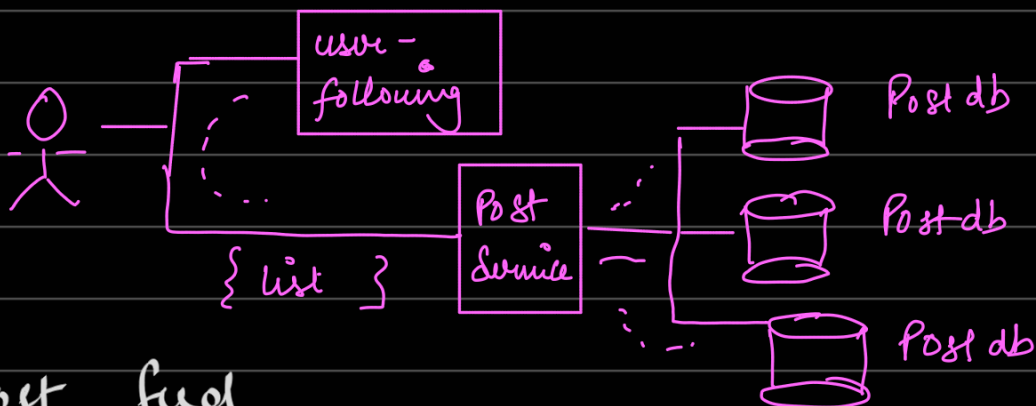


Post feed

(Simple)

- 1) create post
- 2) find following (userid)
- 3) send notification (not-applicable)
- 4) all my followers (will able to see my post)

Reader View



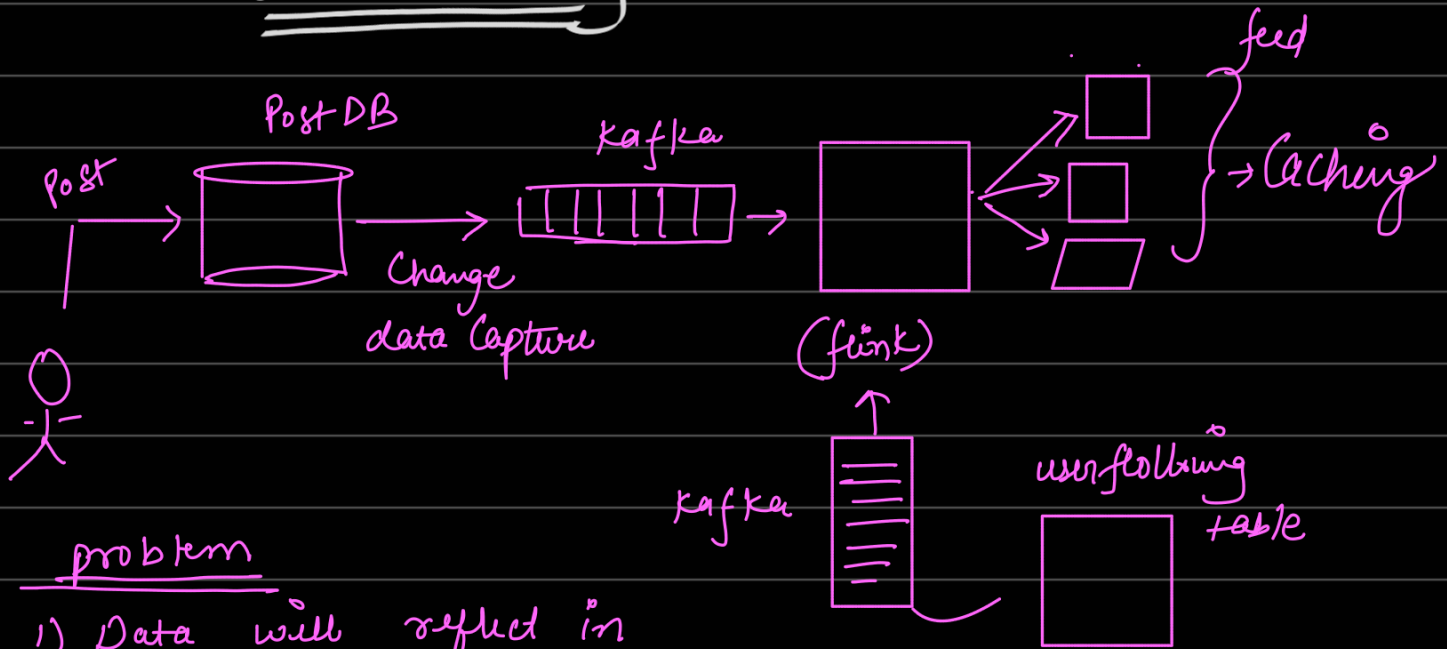
Post feed

(Optimal)

- 1) index post → (from where they belong)
- 2) 1 post > but < 1000 - feed >

- 2) 1 post $\rightarrow$ 200 bytes $\rightarrow$ 1000 feeds
- 3) 1 billion post/day $\cdot$ 200 bytes $\cdot$ 100 copies
 $= 20 \text{ Tb}$
- 4) caching $\rightarrow$ feeds $\rightarrow 20 \text{ Tb} / 256 \text{ Gb}$

Understanding



problem

- 1) Data will reflect in feed little slow [is it okay]?

partition post DB $\rightarrow$ (sql vs nosql)

userid	Timestamp	Post Data
69	01/06/2028	~~~~~
70		
71		

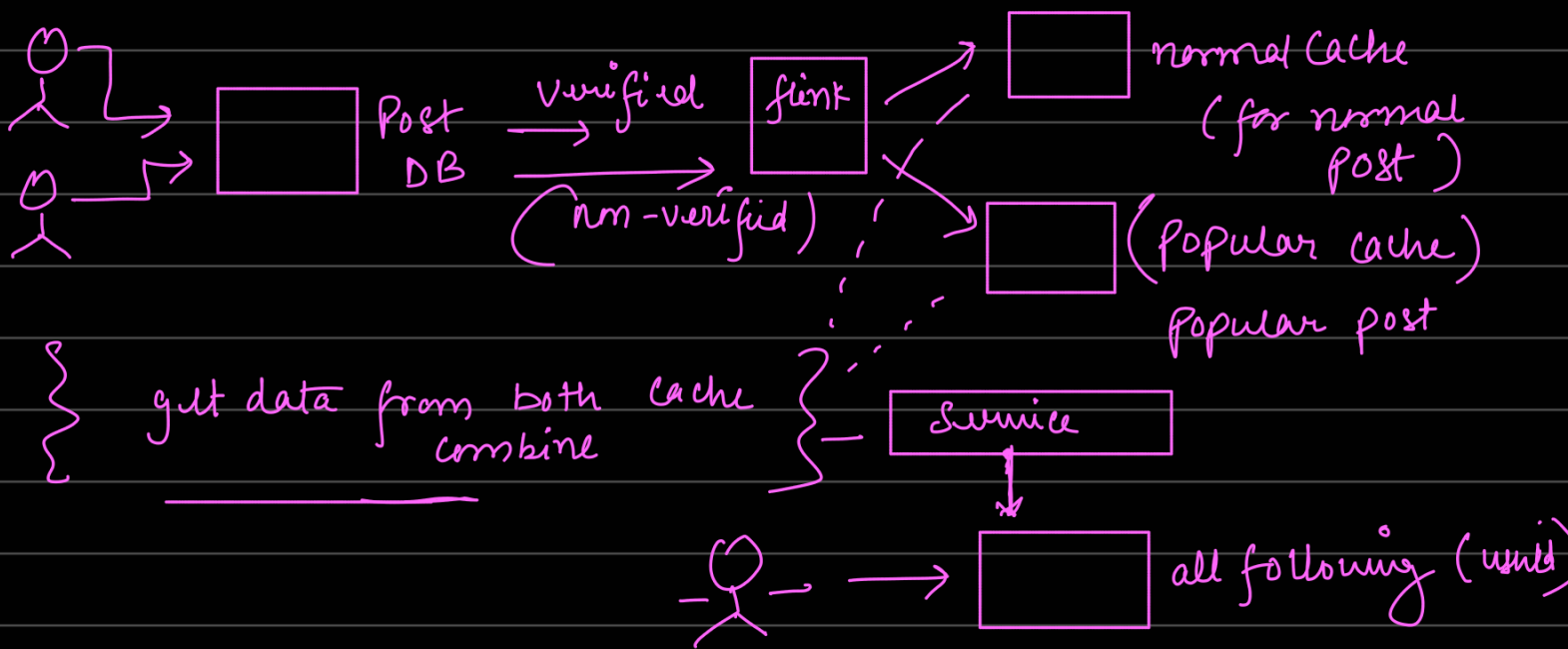
- 1) Partition by userid
- 2) Sorting

Problems of Popular Users

- 1) million of followers
- 2) we have to change tot of cache

solution

- 1) if we know that user is popular.
- 2) lets use two separate cache.



Access level

user - follower		
id	follower id	access
1	pqr	all/ close

User DB

Post		
id	Post data	access
pqr	---	all
pqr	===	close

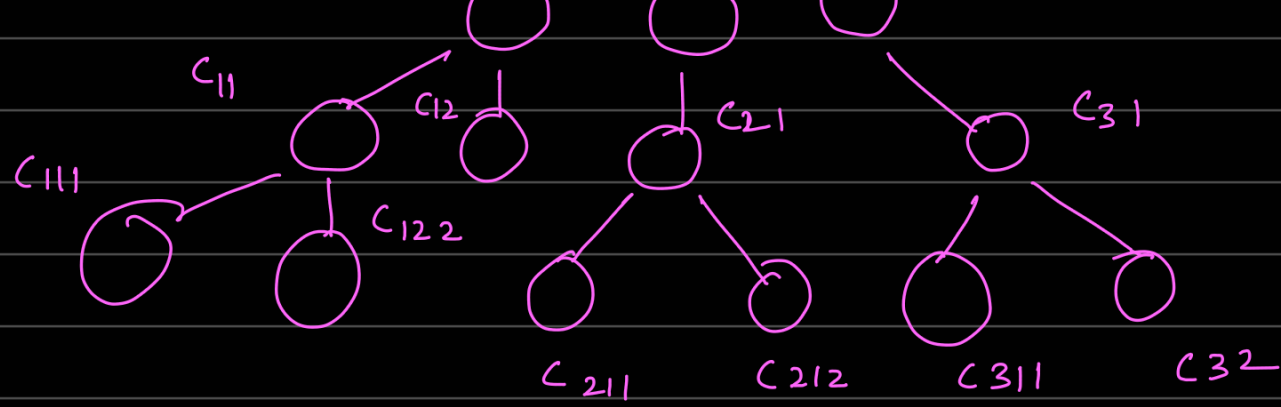
Post

(match)

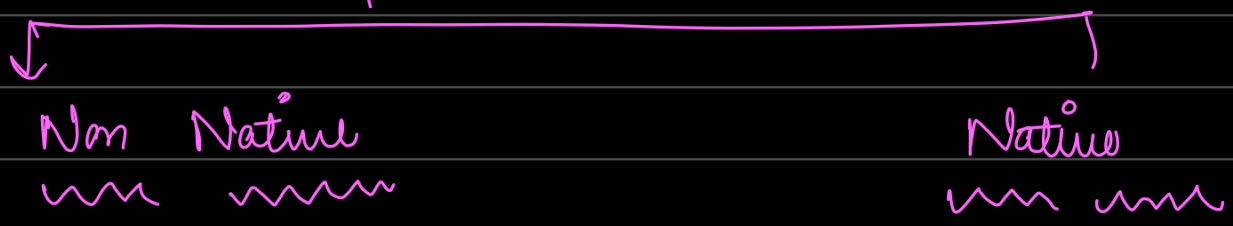
! -> change cache feed

NESTED COMMENTS

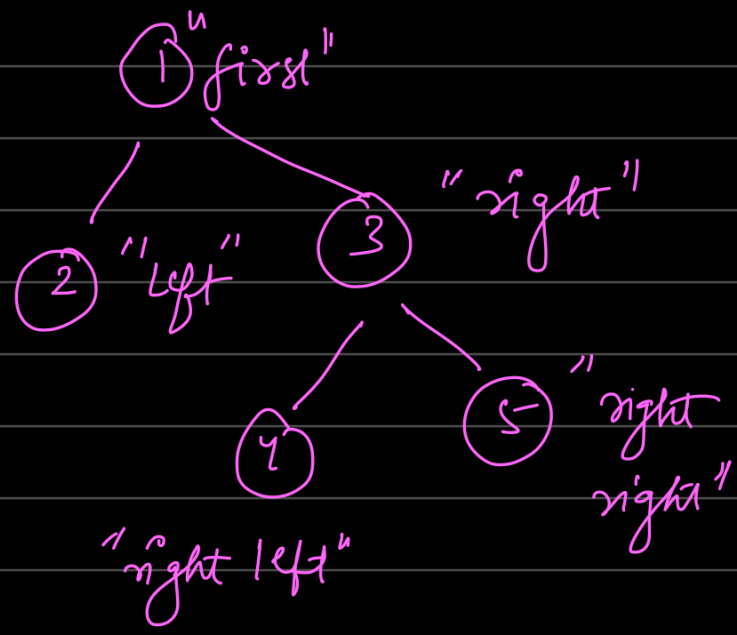




Graph Database (mysql) (DFS)



id	ParentId	Comment
1	null	"~"
2	1	"left"
3	1	"right"
4	3	"~"
5	3	"~"



- 1) Binary Search
- 2) as it grows it will become slow.

- 1) Disk-follower
- 2) slow

